

Homework 1: Divide and Conquer

Guideline

Homework chatbot. [Open the homework Gem here](#). Use this Gem when working through the AI-related sections. The bot is already curated with the right answers and hints to provide the right level of guidance to help you learn.

Showing your honest attempt can only help. There is no advantage in cheating. I give 100% credit on attempts, even if they are incorrect. Cheating on homework will also only hurt you. If you cheat, I won't be able to give you the proper feedback to help you learn, nor adjust the difficulty of the quizzes. You will also likely fail on the in-person mastery quizzes, which test concepts tackled in the homework.

Collaboration: Only after solo attempt. You may collaborate and use external resources, but (1) you are encouraged to do the homework **alone first** and (2) you must **attribute** these external resources when you do use them.

AI: Follow guideline. The problems will be very clear on when you are allowed to use AI. Please follow them. Here is why: (1) [Unregulated usage of AI harms learning](#) (2) Even in the age of AI, your [AI-independent ability matters](#). The **only** AI tool you should use on this homework is the homework-specific [Gemini Gem](#) that I have created for you.

Submission. Upload all files to the Homework 1 assignment on Canvas. **7 files total:**

1. **One answer document** containing your answers to every question below. Name it `lastname-hw1.pdf` (or `.txt`). LaTeX isn't required -- plain text, Google Docs → Download as PDF, or Word → Save as PDF are all fine.
2. **Six AI chat transcript PDFs**, one for each AI-check step. Start a **new chat** in the Gem for each step, so that each transcript covers only that step:
 - `power-b.pdf` , `power-d.pdf`
 - `mountain-peak-b.pdf` , `mountain-peak-d.pdf`
 - `max-subarray-b.pdf` , `max-subarray-d.pdf`

Accepted file formats: `.pdf` for everything; `.txt` is also fine for the answer document. Do **not** paste chat content into your answer document, and do **not** submit a link to a Google Doc -- upload the file itself.

Due: Sunday, September 27, 2026 at 11:59pm. See the syllabus for the late-day policy.

Points: 41 total. Remember, I give full points to every question that was attempted, and 0 only for a question left blank. What matters is that you have **thought** through the problems enough to prepare yourself for the quizzes.

1. Binary search recap (6 pts total)

Do **not** use AI for this section. Refer to the lecture materials and test your mental model.

Let's re-derive the correctness and runtime properties of binary search: `int binary_search(int left, int right, int[] arr, int target)`. Given an array `arr` sorted in ascending order, the function searches the range `arr[left..right]` (inclusive) for `target`, and returns the index of `target` if it is found, or `-1` if it is not.

(a) Correctness

(i) Function specification (1 pt)

Write the function specification for a recursive binary search. Provide all of these:

- The **input** variable name, data type, and an intuitive description of each variable
- The **conditions** that the input variables have to satisfy
- The **output** variable name, and an intuitive description of what it represents

(ii) One-line function specification (1 pt)

Now, in **one line**, write an intuitive function specification that captures all of the above. This is your mental model of what the function does. You can sacrifice completeness for intuitiveness. E.g.

`f(a,b,c)` computes the number `k` such that `a[b...c]` is the most beautiful.

(iii) Pseudocode / recurrence relation (1 pt)

Show how a parent invocation is dependent on smaller, child invocations. That is, what recursive call(s) does `binary_search(l, r, ...)` make, and how is the parent's answer obtained from the child's result? Answer by providing a pseudocode.

(iv) Intuitive proof (1 pt)

Write between 2 and 5 sentences to argue why your recurrence relation is correct.

(b) Runtime

(i) Big-O (1 pt)

What is the big-O runtime of binary search for an array of size `N`?

(ii) Explain (1 pt)

Write between 2 and 5 sentences on how you derive the answer. Use the following logic: "There are `X` levels of recursion, and at each level we have a constant work of `Y`, so the total work is `Z`"

2. Merge sort recap (6 pts total)

Do **not** use AI for this section. Refer to the lecture materials and test your mental model.

We will now re-derive the correctness and runtime properties of merge sort: `merge_sort(int left, int right, int[] arr)`. The function sorts the range `arr[left..right]` (inclusive) in ascending order, in place, and returns nothing.

(a) Correctness

(i) Function specification (1 pt)

Write the function specification for merge sort. Provide all of these:

- The **input** variable name, data type, and an intuitive description of each variable
- The **conditions** that the input variables have to satisfy
- The **post-condition** on any of the input variable after the function call has completed. Note that we do this instead of describing the return value.

(ii) One-line function specification (1 pt)

Now, in **one line**, write an intuitive function specification that captures all of the above. This is your mental model of what the function does. You can sacrifice completeness for intuitiveness. E.g.

`f(a,b,c)` computes the number `k` such that `a[b...c]` is the most beautiful.

(iii) Pseudocode / recurrence relation (1 pt)

Show how a parent invocation is dependent on smaller, child invocations. That is, what recursive calls does `merge_sort(...)` make, and how do you combine the two sorted halves so that you fulfill the post-condition? Answer by providing a pseudocode.

(iv) Intuitive proof (1 pt)

Write between 2 and 5 sentences to argue why your recurrence relation is correct.

(b) Runtime

(i) Big-O (1 pt)

What is the big-O runtime of merge sort for an array of size `N`?

(ii) Explain (1 pt)

Write between 2 and 5 sentences on how you derive the answer. Use the following logic: "There are `X` levels of recursion, and at each level the total work across all sub-calls is `Y`, so the total work is `X·Y`"

3. Power (9 pts total)

Implement `pow(x, n)`, which calculates `x` raised to the power `n`. See the [leetcode link](#) for examples.

(a) No-AI-attempt planning

Understand the problem statement on leetcode first, then start a stop watch for **15 minutes** to tackle this on your own. This is to simulate a real interview where you explain your plan to the interviewer, before you dive into coding. Use the full 15 minutes. If you give up before you have a complete answer, that's fine -- an incomplete attempt still earns full credit.

(i) Function specification (1 pt)

Write the function specification. Note: describe your **recursive helper method**, which may need parameters that the original function's signature doesn't have. Provide all of these:

- The **input** variable name, data type, and an intuitive description of each variable
- The **conditions** that the input variables have to satisfy
- The **output** variable name, and an intuitive description of what it represents

(ii) One-line function specification (1 pt)

Now, in **one line**, write an intuitive function specification that captures all of the above. This is your mental model of what the function does. You can sacrifice completeness for intuitiveness. E.g.

`f(a,b,c)` computes the number k such that `a[b...c]` is the most beautiful.

(iii) Solution sketch (1 pt)

Write between 2 and 5 sentences on the intuition of the recurrence relation that you will use. You would want to think of a solution that is better than $O(n)$

(iv) Runtime (1 pt)

What's the big-O runtime of your solution? Why?

(b) AI plan check

Use the homework chatbot introduced at the beginning of this homework writeup to validate your answers from the previous section. The chatbot is already equipped with the right solution and hints to guide you. Here are some example prompts:

- **Verifying.** "Here is the problem statement (copy paste). Here is my function specification (specification). Is this right?"
- **Brainstorming.** "Could we have done better?"
- **Asking for clarification.** "I don't understand why this pseudocode line matters."

(i) Chat transcript (1 pt)

Turn in a **chat transcript**. In the upper right corner of the gem, you can click "Download PDF". Keep this PDF and include in your submission. Name the PDF "power-b.pdf". Do **not** copy paste the chat content here.

(ii) Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings?
You can also say "Nope, I got everything right"

(c) Code (1 pt)

Start a stop watch for **30 minutes** and code this on leetcode without using AI. Imagine that this is you having to write code on the whiteboard for an interview, and you are under time pressure.

If you have exceeded 30 minutes, you **may** use the homework AI to help you succeed and get a successful leetcode submission.

Turn in your final code, pasted into your answer document. Also paste your Leetcode submission URL showing a successful submission.

(d) AI code check

Copy paste your code to the homework gem and ask for AI review.

Here is an example prompt: "Here is the problem statement (copy) and here is my code (copy). Q1. Can you evaluate for correctness? Q2. Can you help improve my code style? "

(i) Chat transcript (1 pt)

Turn in the chat transcript PDF. Name the PDF "power-d.pdf". Do **not** copy paste the chat content here.

(ii) Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings?

You can also say "Nope, I got everything right"

4. Find peak in mountain (10 pts total)

Implement `peakIndexInMountainArray(int[] arr)`, which returns the index of the peak in a mountain array -- an array whose values strictly increase to a single peak, then strictly decrease. See the [leetcode link](#) for examples.

(a) No-AI-attempt planning

Understand the problem statement on leetcode first, then start a stop watch for **15 minutes** to tackle this on your own. This is to simulate a real interview where you explain your plan to the interviewer, before you dive into coding. Use the full 15 minutes. If you give up before you have a complete answer, that's fine -- an incomplete attempt still earns full credit.

(i) Function specification (1 pt)

Write the function specification. Note: describe your **recursive helper method**, which may need parameters that the original function's signature doesn't have. Provide all of these:

- The **input** variable name, data type, and an intuitive description of each variable
- The **conditions** that the input variables have to satisfy
- The **output** variable name, and an intuitive description of what it represents

(ii) One-line function specification (1 pt)

Now, in **one line**, write an intuitive function specification that captures all of the above. This is your mental model of what the function does. You can sacrifice completeness for intuitiveness. E.g.

`f(a,b,c)` computes the number `k` such that `a[b...c]` is the most beautiful.

(iii) Solution sketch (1 pt)

Write between 2 and 5 sentences on the intuition of the recurrence relation that you will use. You would want to think of a solution that is better than $O(n)$

(iv) Correctness (1 pt)

Give an intuitive explanation (between 2 and 5 sentences) of why your recurrence relation is correct. Why are each of your branches correct?

(v) Runtime (1 pt)

What's the big-O runtime of your solution? Why?

(b) AI plan check

Use the homework chatbot introduced at the beginning of this homework writeup to validate your answers from the previous section. The chatbot is already equipped with the right solution and hints to guide you. Here are some example prompts:

- **Brainstorming.** "I'm stuck. Can you give me a hint?"
- **Verifying.** "Here is the problem statement (copy paste). Here is my function specification (specification). Is this right?"
- **Asking for clarification.** "I don't understand why this pseudocode line matters."

(i) Chat transcript (1 pt)

Turn in a **chat transcript**. In the upper right corner of the gem, you can click "Download PDF". Keep this PDF and include in your submission. Name the PDF "mountain-peak-b.pdf". Do **not** copy paste the chat content here.

(ii) Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings?
You can also say "Nope, I got everything right"

(c) Code (1 pt)

Start a stop watch for **30 minutes** and code this on leetcode without using AI. Imagine that this is you having to write code on the whiteboard for an interview, and you are under time pressure.

If you have exceeded 30 minutes, you **may** use the homework AI to help you succeed and get a successful leetcode submission.

Turn in your final code, pasted into your answer document. Also paste your Leetcode submission URL showing a successful submission.

(d) AI code check

Copy paste your code to the homework gem and ask for AI review.

Here is an example prompt: "Here is the problem statement (copy) and here is my code (copy). Q1. Can you evaluate for correctness? Q2. Can you help improve my code style? "

(i) Chat transcript (1 pt)

Turn in the chat transcript PDF. Name the PDF "mountain-peak-d.pdf". Do **not** copy paste the chat content here.

(ii) Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings?
You can also say "Nope, I got everything right"

5. Maximum subarray (10 pts total)

Given an integer array `nums`, find the contiguous subarray with the largest sum, and return its sum. **For this problem, you must use an $O(N \log N)$ divide-and-conquer approach.** This problem can actually be solved with and without divide and conquer in $O(N)$, but we will defer this to a future lecture. See the [leetcode link](#) for examples.

(a) No-AI-attempt planning

Understand the problem statement on leetcode first, then start a stop watch for **15 minutes** to tackle this on your own. This is to simulate a real interview where you explain your plan to the interviewer, before you dive into coding. Use the full 15 minutes. If you give up before you have a complete answer, that's fine -- an incomplete attempt still earns full credit.

(i) Function specification (1 pt)

Write the function specification. Note: describe your **recursive helper method**, which may need parameters that the original function's signature doesn't have. Provide all of these:

- The **input** variable name, data type, and an intuitive description of each variable
- The **conditions** that the input variables have to satisfy
- The **output** variable name, and an intuitive description of what it represents

(ii) One-line function specification (1 pt)

Now, in **one line**, write an intuitive function specification that captures all of the above. This is your mental model of what the function does. You can sacrifice completeness for intuitiveness. E.g.

`f(a,b,c)` computes the number `k` such that `a[b...c]` is the most beautiful.

(iii) Solution sketch (1 pt)

Write between 2 and 5 sentences on the intuition of the recurrence relation that you will use. Think about how the maximum subarray of the whole array relates to the maximum subarrays of the left half, the right half, and subarrays that cross the middle.

(iv) Correctness (1 pt)

Give an intuitive explanation (between 2 and 5 sentences) of why your recurrence relation is correct. In particular, why do your cases capture every possible maximum subarray?

(v) Runtime (1 pt)

What's the big-O runtime of your solution? Why?

(b) AI plan check

Use the homework chatbot introduced at the beginning of this homework writeup to validate your answers from the previous section. The chatbot is already equipped with the right solution and hints to guide you. Here are some example prompts:

- **Brainstorming.** "I'm stuck. Can you give me a hint?"
- **Verifying.** "Here is the problem statement (copy paste). Here is my function specification (specification). Is this right?"
- **Asking for clarification.** "I don't understand why this pseudocode line matters."

(i) Chat transcript (1 pt)

Turn in a **chat transcript**. In the upper right corner of the gem, you can click "Download PDF". Keep this PDF and include in your submission. Name the PDF "max-subarray-b.pdf". Do **not** copy paste the chat content here.

(ii) Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings?
You can also say "Nope, I got everything right"

(c) Code (1 pt)

Start a stop watch for **30 minutes** and code this on leetcode without using AI. Imagine that this is you having to write code on the whiteboard for an interview, and you are under time pressure.

If you have exceeded 30 minutes, you **may** use the homework AI to help you succeed and get a successful leetcode submission.

Turn in your final code, pasted into your answer document. Also paste your Leetcode submission URL showing a successful submission.

(d) AI code check

Copy paste your code to the homework gem and ask for AI review.

Here is an example prompt: "Here is the problem statement (copy) and here is my code (copy). Q1. Can you evaluate for correctness? Q2. Can you help improve my code style? "

(i) Chat transcript (1 pt)

Turn in the chat transcript PDF. Name the PDF "max-subarray-d.pdf". Do **not** copy paste the chat content here.

(ii) Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings?
You can also say "Nope, I got everything right"